Ingenieria de Datos II · UADE

Plataforma de Logistica Internacional y Monitoreo IoT

Trabajo Practico: Persistencia Poliglota

Resumen completo del sistema

Que hicimos, como funciona y por que lo decidimos asi

Documento de referencia para la defensa individual

Entrega: 17/11/2026 · Congelamiento de codigo: 6/11/2026

Como usar este documento

Esto no es el informe de entrega: es el **manual interno del grupo**. Esta escrito para que cualquiera de los cuatro pueda sentarse frente al profesor y explicar cualquier parte del sistema, incluidas las que no programo.

La defensa es **individual** y tiene tres partes: que la aplicacion funcione en vivo, preguntas sobre las decisiones y como se implemento cada cosa, y posiblemente una modificacion o una consulta nueva pedida en el momento. Por eso el documento insiste menos en *que* hace cada cosa y mas en **por que esta hecha asi y que alternativa se descarto**: eso es lo que se pregunta.

> **Regla para estudiarlo:** si podes responder "por que esta base y no otra" y "que pasaria si lo hubieramos hecho al reves" para cada pieza, estas listo. Las respuestas de memoria se notan; los motivos no.

Contenido

1. De que trata el trabajo
2. La idea central: persistencia poliglota
3. Las cinco bases y por que cada una
4. Como esta organizado el codigo
5. El problema de la doble escritura y el Outbox
6. Las consultas que ninguna base puede responder sola
7. Decisiones clave, agrupadas por tema
8. Como se corre el sistema
9. Que hay dentro de la aplicacion
10. Errores reales que encontramos (y que ensenan)
11. Limitaciones conocidas
12. Preguntas probables y como responderlas

1. De que trata el trabajo

El sistema modela una **plataforma de logistica internacional con monitoreo IoT**. En palabras simples: una empresa que mueve carga en contenedores por barco, camion y avion, y que ademas tiene sensores dentro de esos contenedores reportando temperatura, humedad, vibracion y posicion GPS.

Las cosas que el sistema maneja

Concepto	Que es
Cliente	Quien contrata el envio. Tiene cuenta corriente.
Operador logistico	Quien efectivamente mueve la carga: naviera, transportista, aerolinea.
Proveedor	Quien presta un servicio: estiba, combustible, mantenimiento.
Envio	El pedido completo de un cliente, de origen a destino.
Tramo	Cada etapa del envio. Un envio Buenos Aires → Rotterdam puede tener un tramo terrestre hasta el puerto y uno maritimo hasta destino.
Contenedor	La unidad fisica de carga. Viaja en uno o varios envios a lo largo del tiempo.
Sensor	El equipo IoT instalado en un contenedor. Reporta cada cierto intervalo.
Lectura	Cada medicion que manda un sensor. Es lo que mas crece de todo el sistema.
Incidente	Algo que salio mal en un envio: demora, dano, robo, problema de aduana.
Alerta	Se genera sola cuando una lectura sale del rango permitido.
Factura / Cuenta corriente	Lo que se le cobra a clientes, operadores y proveedores.

Lo que pide el enunciado

El profesor dio una lista de **41 puntos que deben estar "presentes y operativos, es decir que tienen datos para dar un resultado"**. Esa frase es importante y define el criterio de todo el trabajo:

> **Una funcion sin menu no cuenta.** No alcanza con que el codigo exista y tenga tests: si el profesor no la puede ejecutar desde la aplicacion y ver un resultado, vale cero. Buena parte del trabajo final fue justamente **hacer alcanzable** codigo que ya funcionaba pero que nadie podia correr.

Los 41 puntos cubren cuatro familias: los **ABM** (altas, bajas y modificaciones de cada entidad), la **facturacion y cuentas corrientes**, el **monitoreo IoT** (lecturas, alertas, GPS, estado de sensores) y los **reportes analiticos** (rankings, promedios, tiempos, prediccion).

2. La idea central: persistencia poliglota

Persistencia poliglota significa usar varias bases de datos en un mismo sistema, eligiendo cada una segun la forma del dato que guarda, en vez de forzar todo dentro de un unico motor.

La pregunta que hay que saber responder no es "que bases usamos" sino **"por que esta y no otra"**. Y la respuesta correcta nunca es "porque el TP lo pedia": siempre hay una caracteristica del dato que lo justifica.

El razonamiento, en una linea por motor

Si el dato...	Va en	Porque
tiene relaciones estrictas y necesita transacciones	PostgreSQL	Es el unico que garantiza que varias escrituras pasen todas o ninguna
cambia de forma segun el caso	MongoDB	Un documento admite campos distintos sin columnas vacias
es una serie temporal que crece sin parar	Cassandra	Escribe muy rapido y reparte por particiones; no se degrada al crecer
vale por como se conecta con otros	Neo4j	Recorrer relaciones es su operacion natural, no un JOIN encadenado
es temporal y se consulta a cada rato	Redis	Vive en memoria y vence solo, sin que nadie lo borre

> **La contracara, y hay que decirla:** la persistencia poliglota **no es gratis**. Se pierde la transaccion que abarca todo, se pierden las claves foraneas entre motores, y hay que mantener cinco modelos en lugar de uno. Casi todas las decisiones dificiles del trabajo salen de ahi: el Outbox, el chequeo de consistencia, las validaciones en los servicios.

3. Las cinco bases y por que cada una

PostgreSQL 16 — la fuente de verdad

Guarda **17 tablas**: clientes, usuarios, roles, operadores, proveedores, ubicaciones, vehiculos, envios, tramos, contenedores, sensores, la relacion envio-contenedor, tarifas, facturas, pagos, cuentas corrientes y movimientos. Mas una tabla tecnica, `outbox`, que se explica en la seccion 5.

Por que: estos datos tienen relaciones que no pueden romperse (un envio sin cliente no existe) y operaciones que deben ser atomicas. El caso que lo justifica solo es **emitir una factura**: hay que escribir la factura, el movimiento en el libro mayor y el saldo de la cuenta. Si una de las tres falla, ninguna debe quedar. Eso es una transaccion, y solo PostgreSQL la da.

Las reglas del negocio estan declaradas en la base con `CHECK`, no solo en el codigo. Por ejemplo, que un envio `ENTREGADO` tenga fecha de llegada real y uno no entregado no la tenga:

```
CONSTRAINT chk_envio_llegada_segun_estado CHECK (
    (estado = 'ENTREGADO' AND fecha_llegada_real IS NOT NULL) OR
    (estado <> 'ENTREGADO' AND fecha_llegada_real IS NULL))
```

MongoDB 7 — lo que cambia de forma

Guarda cinco colecciones:

Coleccion	Que guarda	Por que documental
`incidentes`	Problemas de un envio, con una foto del envio embebida y su historial de estados	El historial es una lista que se lee siempre junto al incidente
`alertas`	Violaciones de umbral, con el detalle de que valor se paso y de cuanto	El detalle cambia segun el tipo de alerta
`umbrales`	Rango normal de cada magnitud, por tipo de contenedor	Configuracion chica y de forma variable
`reportes`	Solicitudes de reportes parametrizados y facturables	Los parametros cambian segun el reporte: rango, limite, eje
`conversaciones` y `mensajes`	Mensajeria con clientes, proveedores y operadores	Texto libre, sin esquema rigido

> **La pregunta mas probable sobre MongoDB:** "por que el historial de un incidente va *embebido* y los mensajes de una conversacion *no*". La respuesta es **cuanto crece cada uno**. El historial son unos pocos cambios de estado y se lee siempre junto al incidente. Los mensajes de un hilo crecen sin limite, y un documento de MongoDB no puede pasar los **16 MB**. Por eso los mensajes son una coleccion aparte que referencia su conversacion.

Cassandra 5 — la serie temporal

Guarda las **lecturas de los sensores**, el **recorrido GPS** de los contenedores y la **traza de eventos** de cada envio.

Por que: es el dato que mas crece. Con cinco sensores reportando cada minuto son 7.200 filas por dia; con quinientos, 720.000. Cassandra esta hecha para escribir mucho y rapido, y reparte los datos en particiones para que consultar no se vuelva mas lento a medida que crece.

Lo que hay que entender de Cassandra: se disena *al reves* que una base relacional. Primero se escribe la consulta que uno quiere hacer, y despues se crea una tabla a medida de esa consulta. Por eso el mismo dato esta repetido en varias tablas: `lecturas_por_sensor` y `recorrido_por_contenedor` guardan partes de la misma lectura, porque se consultan distinto.

La **clave de particion** de cada tabla incluye el dia (`(sensor_id, dia)`). Eso evita que un sensor con anos de historia termine con una particion gigante. La consecuencia practica, que aparece en la aplicacion: pedir un rango de fechas son varias consultas, una por dia, y por eso hay un tope de **31 dias por consulta**.

Neo4j 5 — la red de relaciones

Guarda una **proyeccion** de la red logistica: clientes, envios, tramos, contenedores, operadores, vehiculos y ubicaciones, con las relaciones entre ellos.

Por que: hay preguntas que en SQL serian una cadena de JOIN incomoda y en un grafo son un recorrido natural. Las tres que resuelve:

- la **ruta completa** de un envio, tramo por tramo;
- la **red de un cliente**: sus envios y que operadores los movieron;
- por que **tramos paso un contenedor**.

> **Dato importante:** Neo4j **no es duenio de nada**. Todo lo que tiene es derivado de PostgreSQL. Si se borrara entero, se puede reconstruir con una opcion del menu. Eso es lo que hace defendible que su consistencia sea eventual.

Redis 7 — lo temporal

Cumple cuatro funciones, y cada una usa una estructura distinta:

Uso	Estructura	Por que Redis
Sesiones de usuario	Hash con TTL	La sesion nace con fecha de muerte: Redis la borra sola al vencer
Heartbeat de sensores	Sorted set	"Cuanto hace que no reporta" es una sola lectura, no un barrido
Cache de reportes	String con TTL de 60s	Consultas pesadas que varios piden con los mismos parametros
Locks de procesos	`SET NX EX`	Evita que dos instancias corran el mismo proceso a la vez

4. Como esta organizado el codigo

El codigo esta en **capas**, y la regla es que las de adentro no saben nada de las de afuera.

```
domain/          Las entidades y sus reglas. No sabe que existen las bases.
repositories/    Los contratos (clases abstractas): "que se puede guardar y pedir".
infrastructure/  Una carpeta por motor. Implementa los contratos.
services/        Los casos de uso. Orquesta: valida, compone, coordina.
sync/            El Outbox y los proyectores hacia Neo4j y Cassandra.
jobs/            Procesos que corren solos: simulador IoT y planificador.
cli/             Los menus de la aplicacion.
container.py     Arma todo y conecta las piezas.
```

Por que esta division

El dominio no conoce las bases. Un `Envio` sabe que no puede pasar de `CREADO` a `ENTREGADO` sin pasar por el medio, pero no sabe que existe PostgreSQL. Esa regla se cumple igual venga de donde venga el dato.

Los repositorios son contratos, no clases concretas. El servicio pide un `EnvioRepository`; que detras haya PostgreSQL es un detalle. Por eso los tests pueden usar dobles en memoria y correr sin ninguna base.

Tres patrones que conviene saber nombrar

Repository

Cada entidad tiene una interfaz que define que operaciones se pueden hacer. Lo importante: **los repositorios reciben y devuelven entidades del dominio, nunca filas ni diccionarios**. Si un repositorio devolviera un diccionario, el conocimiento de como esta guardado el dato se escaparia hacia arriba.

Y **no existe un metodo** `cambiar_estado(id, estado)`. Si existiera, se podria saltear la validacion de transiciones. Para cambiar un estado hay que traer la entidad, pedirle el cambio a ella (que valida) y guardarla.

Unit of Work

Es lo que permite que varias escrituras compartan una transaccion. Todos los repositorios de PostgreSQL comparten la misma conexion:

```
with self._uow as uow:
    uow.facturas.guardar(factura)
    uow.movimientos.guardar(movimiento)
    uow.cuentas.actualizar(cuenta)
    uow.commit()          # las tres, o ninguna
```

Si algo falla antes del `commit`, al salir del `with` se hace `rollback` automaticamente.

Inyeccion de dependencias

`container.py` es el unico lugar donde se decide que implementacion concreta usa cada servicio. Nada mas en el sistema crea una conexion.

5. El problema de la doble escritura y el Outbox

Esta es **la pieza mas importante del trabajo** y la que mas probablemente pregunten. Vale la pena que los cuatro la sepan explicar.

El problema

Cuando se crea un envio hay que guardarlo en PostgreSQL *y* reflejarlo en el grafo de Neo4j. Son dos bases distintas: **no comparten transaccion**. Escribir en las dos, una despues de la otra, tiene un agujero:

```
guardar en PostgreSQL   -> OK
guardar en Neo4j        -> falla (se cayo la red)
# resultado: el envio existe pero el grafo no lo tiene. Nadie se entera.
```

Y al reves es peor: si primero se escribe Neo4j y falla PostgreSQL, queda un envio en el grafo que no existe. Esto se llama **problema de la doble escritura** (*dual write*).

La solucion: el patron Outbox

En vez de escribir en las dos bases, se escribe **solo en PostgreSQL**: el dato y, en la misma transaccion, un **evento** en una tabla llamada `outbox`.

```
with uow:
    uow.envios.guardar(envio)                      # el dato
    uow.outbox.guardar(eventos.envio_creado(envio))  # el aviso
    uow.commit()      # las dos, o ninguna: es UNA transaccion
```

Despues, un proceso aparte (el **worker**) lee los eventos pendientes y los aplica en Neo4j y en Cassandra. Si falla, el evento queda pendiente y se reintenta.

> **La frase que resume todo:** el Outbox convierte dos escrituras que podian desincronizarse en **una sola escritura transaccional mas una propagacion reintentable**.

Los tres detalles que hacen que funcione

1. El evento lo publica el servicio

No el repositorio ni un trigger de la base. El repositorio no sabe *por que* se guarda algo, y un trigger esconderia logica de negocio dentro de la base, donde nadie la busca ni se puede testear.

2. Los proyectores son idempotentes

Como un evento se puede reintentar, puede aplicarse dos veces. Por eso las consultas Cypher usan `MERGE` (crea si no existe, no duplica) y **nunca** `CREATE`.

```
MERGE (e:Envio {id: $id})      // si ya esta, no hace nada
SET e.estado = $estado
```

La combinacion de **reintento** (entrega al menos una vez) con **idempotencia** da el efecto de "exactamente una vez".

3. Hay reintentos con espera creciente

Un evento que falla no se reintenta inmediatamente para siempre: espera cada vez mas (*backoff exponencial*) y hay un tope de intentos. Si no, un evento roto consumiria el worker eternamente.

La red de seguridad

Igual puede pasar algo raro. Por eso hay dos herramientas en el menu de Sincronizacion:

- **Verificar consistencia:** compara que cada envio, cliente y contenedor de PostgreSQL tenga su nodo en Neo4j, y lista los que falten.
- **Reconstruir el grafo:** borra Neo4j entero y lo rehace desde PostgreSQL. Se puede hacer porque el grafo no es duenio de nada.

Decisiones relacionadas: D34 (quien publica), D35 (idempotencia), D36 (chequeo de consistencia), D40 (la traza de Cassandra tambien pasa por el Outbox).

6. Las consultas que ninguna base puede responder sola

Estas son las piezas que **demuestran** que el sistema es poliglota de verdad y no cinco bases puestas al lado. Si el profesor pregunta "para que les sirvio usar varias bases", la respuesta esta aca.

Promedio de incidentes por pais, estado o cliente

Los incidentes estan en MongoDB y los envios en PostgreSQL. **MongoDB cuenta el numerador** (cuantos incidentes hay por grupo) y **PostgreSQL el denominador** (cuantos envios hay en ese grupo). El promedio lo compone el servicio, porque ninguna consulta sola lo responde.

> **El caso mas interesante de todo el TP:** el promedio **por estado del envio**. Ese estado *no esta* en el documento del incidente, y no puede estarlo: el incidente guarda una foto de cuando ocurrio, y el estado del envio **cambia con el tiempo**. Entonces MongoDB agrupa los incidentes por envio, PostgreSQL dice en que estado esta hoy cada uno, y el servicio los suma por estado.

Ranking de rutas con mas incidentes

Este es el contrario, y por eso es un buen par para comparar: se resuelve **entero dentro de MongoDB**, con una sola agregacion. Funciona porque cada incidente guarda embebida una **foto de la ruta** (`ARBUE->NLRTM`) al momento en que ocurrio. Sin esa foto habria que reconstruir la ruta desde el grafo o desde los tramos para cada incidente.

La moraleja: una denormalizacion bien elegida convierte una consulta que cruza tres motores en una sola agregacion.

Reportes solicitados y facturables

Un flujo de tres pasos que toca tres motores:

Paso	Que pasa	Donde
1. Solicitar	Se registra el pedido con sus parametros y se le pone precio	MongoDB
2. Generar	Se calcula el reporte y se guarda cuantas filas dio	PostgreSQL / MongoDB
3. Facturar	Se emite la factura y se imputa a la cuenta corriente	PostgreSQL

Por que son tres operaciones y no una: pueden fallar por separado y pasan en momentos distintos. Si fuera un solo metodo, un error al calcular borraria el pedido del cliente.

El precio queda congelado en la solicitud: si manana cambia la tarifa, se factura lo que se cotizo cuando el cliente lo pidio.

Prediccion de atrasos

Cruza PostgreSQL (envios y tramos) con MongoDB (incidentes abiertos) para armar, por cada envio en curso, un **puntaje de riesgo con sus motivos** y una fecha de llegada proyectada.

> **No se uso machine learning, y hay que defenderlo:** con seis envios de prueba un modelo no tendria de donde aprender. Pero el motivo de fondo es otro: ante "por que este envio es de riesgo ALTO" la unica respuesta seria "porque lo dijo el modelo". **Un sistema de logistica que no puede explicar una alerta no sirve para operar.**

Los factores que suman puntos, todos observables:

Factor	Puntos	De donde sale
Tramos que ya llegaron tarde	2 c/u	PostgreSQL
Tramos vencidos sin registrar llegada	3 c/u	PostgreSQL
Incidentes abiertos del envio	2 c/u	MongoDB
...de severidad ALTA o CRITICA	+3 c/u	MongoDB
La llegada estimada vencio y falta recorrido	4	PostgreSQL
Recorrido completo sin entrega registrada	3	PostgreSQL

El puntaje mide riesgo; el atraso proyectado mide tiempo. Son dos numeros distintos y no se mezclan: el atraso sale del que el envio ya acumulo en sus tramos, no del puntaje. Derivar dias a partir de puntos daria un numero inventado con cara de dato.

Monitoreo IoT: cuatro motores en una pantalla

Dato	Motor	Por que
Ficha del sensor	PostgreSQL	Dato estable, se relaciona con el contenedor
Lectura	Cassandra	Serie temporal que crece sin parar
Umbral	MongoDB	Configuracion chica y de forma variable
Alerta	MongoDB	Documento con detalle embebido
Heartbeat	Redis	"Cuanto hace que no reporta", en una lectura

7. Decisiones clave, agrupadas por tema

El proyecto tiene **58 decisiones documentadas** en `docs/decisiones.md`. Aca estan las que mas probablemente se pregunten, agrupadas. Cada una dice *que se decidio* y *que se descarto*.

Sobre el dominio

#	Decision	Por que
D3	Donde va cada validacion	**El criterio:** "puedo verificar esta regla con los campos del objeto, sin consultar ninguna base?" Si es si, va en el dominio. Si no, va en el servicio. Ejemplo: que el CUIT tenga 11 digitos va en la entidad; que no este repetido va en el servicio.
D4	Las transiciones de estado viven en el dominio	Un diccionario consultado desde el enum dice a que estados se puede pasar. Se descarto el patron State: mas clases para el mismo mapa.
D6, D7	`Envio` no carga sus tramos; la fecha estimada se deriva	La llegada estimada es el maximo de los tramos. Guardarla seria un dato duplicado que hay que sincronizar cada vez que cambia un tramo.
D15	El dinero es `Decimal`, nunca `float`	Con float, `0.1 + 0.2` no da `0.3` y los saldos se desvian. En la base es `NUMERIC(12,2)`.
D18	Los repositorios no exponen `cambiar_estado`	Si existiera, se podria saltear la validacion de transiciones.
D19	Los valores de los enums son identicos en los cinco motores	Todo en MAYUSCULAS. Si Mongo guardara `"activo"` y Postgres `"ACTIVO"`, cualquier cruce fallaria en silencio.

Sobre la persistencia

#	Decision	Por que
D28	Unit of Work	Los repositorios comparten conexion para que varias escrituras sean una transaccion.
D29	Los errores del motor se traducen	Un `UniqueViolation` de psycopg se convierte en `ConflictoDeDatos`. Asi el servicio no sabe que usa PostgreSQL.
D33	Los reportes devuelven objetos, nunca tuplas	`fila.razon_social` se entiende; `fila[2]` hay que ir a buscarlo.
D41	El saldo se guarda *ademas* del libro mayor	Denormalizacion deliberada: derivarlo sumando todos los movimientos crece sin limite. El riesgo se controla con la transaccion y con `verificar_saldos()`.
D45	El tope de dias de consulta vive en el contrato	Estaba escrito dos veces con semanticas distintas: el servicio medía tiempo transcurrido y el motor días de calendario. Pedir "31 dias" pasaba uno y lo rechazaba el otro.
D56	Las fechas de la semilla son relativas	Con fechas fijas los datos envejecen: ocho meses despues, *todos* los envios figuraban cientos de dias tarde y "envios demorados" dejaba de distinguir nada.

Sobre seguridad

#	Decision	Por que
D27	Contrasenas con PBKDF2-HMAC-SHA256	200.000 iteraciones y salt de 16 bytes por usuario. **Por que no SHA256 pelado:** un hash comun esta hecho para ser rapido, que es justo lo que no se quiere. **Por que el salt:** sin el, dos usuarios con la misma clave tienen el mismo hash y una tabla precalculada los rompe a los dos. Se compara con `hmac.compare_digest` para que tarde siempre lo mismo y no se pueda adivinar byte a byte.
D42	La autenticacion es la puerta	La aplicacion no arma ningun menu hasta que alguien se autentica. Tres intentos y corta. Antes habia usuarios, roles y hashes... y nada de eso protegia nada: `nivel_permisos` no se leia en ningun lado.
D49	El filtro de permisos se reusa en los submenus	Una sola funcion decide que ve cada rol, en los dos niveles del menu. No hay `if` de permisos repartidos.

Sobre las bases NoSQL

#	Decision	Por que
D12	El incidente guarda una foto del envio	Permite el ranking de rutas con una sola agregacion en Mongo. No se actualiza si despues cambia la ruta: registra las circunstancias de cuando ocurrio.
D9	`Tramo` es un *nodo* en Neo4j, no una relacion	Un tramo conecta cuatro entidades (envio, dos ubicaciones y un vehiculo) y una relacion de grafo solo une dos.
D13	El heartbeat usa un sorted set	El score es la marca de tiempo del ultimo reporte: "cuanto hace que no reporta" es una lectura.
D44	La cache nunca rompe el reporte	Si Redis falla al leer o al escribir, se consulta PostgreSQL igual. Un Redis caido hace la aplicacion mas lenta, no la rompe.
D57	Los mensajes van referenciados, no embebidos	Crecen sin limite y embeberlos pasaria los 16 MB por documento.

8. Como se corre el sistema

Los comandos, en orden

```
# 1. Levantar las cinco bases
docker compose up -d

# 2. Verificar que las cinco respondan (muestra la version de cada una)
python scripts/chequear_conexiones.py

# 3. Aplicar el esquema de PostgreSQL y los datos de prueba
python scripts/aplicar_esquema.py

# 4. Arrancar la aplicacion
python -m logistica
```

> **Por que el paso 2 muestra las versiones:** porque un contenedor puede estar "Up" y *no* ser el que recibe las conexiones. Nos paso: el contenedor tenia MongoDB 7.0 pero la aplicacion hablaba con un MongoDB 8.3 instalado en Windows que ocupaba el mismo puerto. Ver la version lo delata.

Usuarios de prueba

La contrasena de los cuatro es `demo1234`.

Email	Rol	Nivel	Que ve
`admin@logistica.com`	ADMINISTRADOR	100	16 de 16 secciones
`carlos.ops@logistica.com`	OPERADOR	60	14 de 16
`lucia.audit@logistica.com`	AUDITOR	30	9 de 16, solo lectura
`mariana@agroglobal.com`	CLIENTE	10	Ninguna

> **Para mostrar en vivo:** entrar con Mariana. Autentica bien y la aplicacion le dice que su rol no tiene acceso a ninguna seccion. Es la prueba de que el control existe, no una pantalla que dice que existe. La consola es una herramienta interna: un cliente no opera el back-office.

Procesos que corren aparte

```
# Simulador IoT: genera lecturas y detecta anomalias al escribirlas
python -m logistica.jobs.simulador_iot --ciclos 5 --intervalo 1

# Planificador: sincronizacion, consistencia, facturacion automatica,
# cierre mensual, verificacion de saldos y deteccion de sensores inactivos
python -m logistica.jobs.planificador
```

9. Que hay dentro de la aplicacion

16 secciones, con **115 opciones** en total. El menu se recorta segun el rol de quien entro.

Seccion	Lo que hace	Motores
Clientes	ABM completo, baja logica	PostgreSQL + Outbox
Contenedores	ABM y cambio de estado administrativo	PostgreSQL + Outbox
Proveedores	ABM completo	PostgreSQL
Operadores logisticos	ABM completo	PostgreSQL
Usuarios, roles y sesiones	ABM de usuarios y roles; sesiones activas	PostgreSQL + Redis
Envios y trayectos	Alta, tramos, estados, contenedores, trazabilidad	PostgreSQL + Cassandra
Facturacion y cuentas	Facturas, pagos, cierre mensual, verificacion de saldos	PostgreSQL
Reportes	7 reportes analiticos, con cache	PostgreSQL + Redis
Consultas de la red logistica	Ruta, red del cliente, movimientos de contenedor	Neo4j
Seguimiento GPS	Recorrido de un contenedor	Cassandra
Incidentes	Alta, estados, ranking de rutas, promedios	MongoDB + PostgreSQL
Monitoreo IoT y alertas	Sensores, lecturas, umbrales, alertas, detecciones	Los cinco
Prediccion de atrasos	Riesgo por envio, con motivos	PostgreSQL + MongoDB
Solicitudes de reportes	Pedir, generar y facturar reportes	MongoDB + PostgreSQL
Mensajeria con las partes	Hilos con clientes, proveedores y operadores	MongoDB + PostgreSQL
Sincronizacion entre motores	Outbox, consistencia, reconstruir el grafo	Los tres derivados

Datos concretos que sirven para la demo

Para mostrar	Usar	Da
Recorrido GPS	Contenedor `HLCU7654321`	150 posiciones
Movimientos en el grafo	`MSKU1234567` o `CMAU9876543`	2 tramos
Envios demorados	Opcion directa	2 de 4 en curso
Prediccion de riesgo	Opcion directa	2 BAJO, 2 ALTO
Alertas abiertas	Opcion directa	91
Cuenta con deuda	Naviera Rio de la Plata	-12.400

10. Errores reales que encontramos (y que ensenan)

Estos casos sirven para dos cosas: entender el sistema de verdad, y tener algo concreto que contar si preguntan "que les costo" o "que aprendieron".

El contenedor que estaba "Up" y no era el que respondia

La aplicacion se conectaba a MongoDB y todo parecia andar, pero los datos no coincidian. El contenedor Docker tenia MongoDB 7.0 y respondia un MongoDB 8.3 instalado en Windows que ocupaba el puerto 27017.

Leccion: que un contenedor este "Up" no significa que sea el que recibe tus conexiones. Por eso el script de verificacion imprime la version.

Los umbrales que cubrian 2 de 7 tipos de contenedor

La deteccion de anomalias salteaba tres de los cinco sensores **en silencio**, porque no habia umbral configurado para su tipo de contenedor. Al completarlos, una corrida identica paso de 14 a 45 alertas.

Leccion: una configuracion incompleta es peor que una ausente, porque parece que esta funcionando. De ahi salio el control de "cobertura de umbrales".

Una relacion del grafo que no existia

Una consulta de Neo4j buscaba una relacion `RECORRE` que ningun proyector creaba. Devolvia vacio sin error. Se reescribio recorriendo las relaciones que si existen.

Leccion: en un grafo, una consulta mal escrita no falla: devuelve nada. Hay que verificar contra datos que uno sepa que existen.

Un metodo que no guardaba nada, en silencio

El `guardar` de alertas es un *insert-if-not-exists*: si la alerta ya existe, no la toca. Esta bien para que un reintento no duplique. Pero al usarlo para *reconocer* una alerta, el cambio de estado no se guardaba y no avisaba.

Leccion: un metodo idempotente para crear no sirve para modificar.

Dar de baja a un usuario apretando Enter

La busqueda hacia `texto in str(id)`, y `""` esta contenido en cualquier cosa: con la entrada vacia se quedaba con el primero de la lista. Apretar Enter por error desactivaba a alguien al azar. Lo encontro una pasada completa por las 115 opciones de la aplicacion.

Leccion: los 582 tests pasaban. Ninguno probaba "que pasa si el usuario aprieta Enter".

Dos modelos para la misma cosa

La coleccion de solicitudes de reportes se implemento sin mirar que el modelo fisico de MongoDB *ya la tenia disenada* con otro nombre y otra forma. Quedaron dos disenos del mismo concepto. Se unificaron.

Leccion: el modelo fisico es parte del codigo, no documentacion. Si se ignora, el validador de esquema te lo recuerda.

11. Limitaciones conocidas

Conviene saberlas y decirlas antes de que las encuentren. Reconocer un limite con su motivo da mas confianza que fingir que no existe.

Limitacion	Por que esta, y que haria falta
Neo4j y Cassandra tienen consistencia eventual	El Outbox propaga despues, no al instante. Es el precio de no tener transaccion entre motores. Se mitiga con el chequeo de consistencia y la reconstruccion.
Marcar un reporte como facturado queda fuera de la transaccion	MongoDB y PostgreSQL no comparten transaccion. Si falla, queda la factura emitida y la solicitud en GENERADA. Por eso existe "pendientes de facturar", que encuentra justo ese caso.
Las contrasenas usan PBKDF2 y no argon2	Para no sumar una dependencia. En un sistema real irian argon2 o bcrypt.
La sesion no se renueva por actividad	Con un TTL de una hora, vence aunque el usuario este trabajando.
La prediccion no mira el historico de la ruta	Seria el factor mas interesante, pero con dos envios entregados el promedio no significaria nada. Es lo primero que agregariamos con datos reales.
La deteccion de anomalias solo evalua temperatura	El enunciado pide temperatura. Sumar humedad o vibracion es agregar una fila a una tabla del codigo.
Los mensajes entrantes se cargan a mano	En un sistema real llegarian por correo o API. La opcion de menu existe para poder mostrar el ida y vuelta completo.
Replicacion y sharding estan documentados, no activos	El Docker es de un solo nodo. Las claves de fragmentacion estan elegidas y justificadas en el modelo.

12. Preguntas probables y como responderlas

Sobre la eleccion de motores

"Por que no hicieron todo en PostgreSQL?"

Se podria, pero cada pieza estaria peor. Las lecturas de sensores crecen sin limite y una tabla relacional se degrada al crecer; los umbrales y los parametros de reportes cambian de forma y serian columnas casi siempre vacias o una columna JSON, que es un documento disfrazado; las consultas de red serian cadenas de JOIN sobre una tabla de tramos; y las sesiones necesitarian un proceso que borre las vencidas, que en Redis lo hace el motor.

"Y por que no todo en MongoDB?"

Por la facturacion. Emitir una factura escribe la factura, el movimiento y el saldo, y las tres tienen que pasar o ninguna. Eso es una transaccion y es exactamente lo que PostgreSQL garantiza.

"Que pierden por usar cinco bases?"

La transaccion que abarca todo y las claves foraneas entre motores. Por eso la existencia de un cliente al guardar un incidente la valida el servicio y no la base, y por eso existe el Outbox.

Sobre la implementacion

"Como se mantienen sincronizadas las bases?"

Con el patron Outbox: el dato y el evento se escriben en una sola transaccion de PostgreSQL, y un worker propaga los eventos a Neo4j y Cassandra con reintentos. Los proyectores son idempotentes (`MERGE`, nunca `CREATE`), asi que reintentar no duplica. Ademas hay un chequeo de consistencia y la posibilidad de reconstruir el grafo entero.

"Que pasa si se cae Redis?"

Depende de para que. La cache de reportes degrada a consultar PostgreSQL, asi que solo se vuelve mas lento. Las sesiones si dependen de Redis: no se puede entrar. Por eso, si las credenciales son correctas pero Redis no responde, la aplicacion lo dice explicitamente en vez de pedir la contrasena de nuevo.

"Por que el estado de un contenedor no se puede poner a mano?"

Los estados `ASIGNADO` y `EN_TRANSITO` los escribe el servicio de envios junto con la fila de `envio_contenedor`, en la misma transaccion. Si el ABM pudiera ponerlos, quedaria un contenedor marcado como ocupado sin ningun envio que lo tenga, y la regla de "un contenedor no puede estar en dos envios activos" empezaria a rechazar asignaciones validas.

"Como evitan que dos instancias corran el mismo proceso?"

Con un lock en Redis (`SET NX EX`): el primero que lo toma corre, el otro se saltea esa vuelta. Ademas los procesos son idempotentes: la facturacion automatica consulta primero si el envio ya tiene factura. La idempotencia protege de la repeticion secuencial, el lock de la concurrencia.

Sobre el modelo

"Por que el incidente guarda una copia de los datos del envio?"

Para que el ranking de rutas se resuelva con una sola agregacion en MongoDB, sin cruzar bases. Y porque un incidente registra las circunstancias de *cuando ocurrio*: si despues cambia la ruta del envio, el incidente tiene

que seguir diciendo donde estaba en ese momento.

"Por que el tramo es un nodo y no una relacion en Neo4j?"

Porque conecta cuatro entidades: el envio, la ubicacion de origen, la de destino y el vehiculo. Una relacion de grafo une exactamente dos nodos.

"Por que repiten datos entre tablas de Cassandra?"

Porque Cassandra se disena por consulta: primero se escribe la pregunta y despues la tabla que la responde. No tiene JOIN, asi que la unica forma de responder dos preguntas distintas sobre el mismo dato es tenerlo guardado de dos formas.

> **Si preguntan algo que no sabes:** conviene decir donde estaria la respuesta ("eso lo resuelve el servicio de X, que cruza tal cosa con tal otra") antes que inventar. Mostrar que entendes la estructura vale mas que acertar un detalle.

Si piden una modificacion o una consulta en vivo

El sistema esta armado para que agregar cosas sea corto, y conviene saber donde toca cada una:

Si piden...	Donde se toca
Un reporte nuevo sobre envios	`ReporteEnvioRepository` (la consulta) + `ReporteService` + una opcion en `menu_reportes`
Un campo nuevo en una entidad	La entidad en `domain/`, la tabla, el `_a_entidad` del repositorio y el `INSERT` / `UPDATE`
Una consulta nueva al grafo	`Neo4jConsultaRepository` (el Cypher) + `ConsultaGrafoService` + `menu_grafo`
Un tipo de alerta nuevo	Una fila en la tabla `_MEDICIONES` de la deteccion de anomalias
Un estado nuevo en el ciclo de un envio	El enum y el mapa de transiciones en `domain/envio.py`, y el `CHECK` de la tabla
Un reporte facturable nuevo	Una entrada en el `CATALOGO` y una rama en `_calcular()`